

Practical no:03

3. Stack Application:

Write a program to:

a. Implement a stack using an array.

Code:

```
#include<iostream>
using namespace std;
int stack[5],top=-1;
void push(int n)
{
    top++;
    if(top<5)
    {
        stack[top]=n;
    }
    else
    {
        cout<<endl<<"Stack is overflow"<<endl;
        top--;
    }
}
void pop()
{
    if(top!=-1)
    {
        top--;
    }
    else
    {
        cout<<endl<<"Stack is Underflow";
    }
}
void peek()
{
    cout<<endl<<"Top:"<<stack[top];
}
void show()
{
    int x;
    cout<<"====Stack===="<<endl;
    for(x=top;x>=0;x--)
    {
        cout<<stack[x]<< endl;
    }
}
int main()
{
    cout<<"After Deletion of element"<<endl;
    pop();
    cout<<endl<<"After Adding element"<<endl;
    push(10);
    push(20);
    push(30);
    push(40);
    push(50);
```

```
        push(60);
        show();
        pop();
        show();
        peek();
        return 0;
    }
```

Output:

After Deletion of element

Stack is Underflow

After Adding element

Stack is overflow

====Stack====

- 50
 - 40
 - 30
 - 20
 - 10
- ====Stack====
- 40
 - 30
 - 20
 - 10

Top:40

b. Convert an infix expression to postfix notation using a stack.

Code:

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;

stack<char> st;
string postfix;

int precedence(char op)
{
    if (op == '+' || op == '-')
        return 1;
    if (op == '*' || op == '/')
        return 2;
    if (op == '^')
        return 3;

    return 0;
}

void infixToPostfix(string infix)
{
    for (int c = 0; c < infix.length(); c++)
    {
        char ch = infix[c];
        if ((ch >= 'A' && ch <= 'Z') ||
            (ch >= 'a' && ch <= 'z') ||
            (ch >= '0' && ch <= '9'))
        {
            postfix += ch;
        }
        else if (ch == '(')
        {
            st.push(ch);
        }
        else if (ch == ')')
        {
            while (!st.empty() && st.top() != '(')
            {
                postfix += st.top();
                st.pop();
            }

            if (!st.empty())
                st.pop();
        }
        else
        {
            while (!st.empty() &&
                precedence(st.top()) >= precedence(ch))
            {
                postfix += st.top();
                st.pop();
            }
        }
    }
}
```

```
        st.push(ch);
    }
}

while (!st.empty())
{
    postfix += st.top();
    st.pop();
}

cout << "Postfix: " << postfix << endl;
}

int main()
{
    string infix = "A+B";

    infixToPostfix(infix);

    return 0;
}
```

Output:

Postfix: AB+

...Program finished with exit code 0
Press ENTER to exit console.█

Practical.No:04

4. Queue Application: Write a program to:

a. Implement a queue using an array.

Code:

```
#include <iostream>
using namespace std;
int queue[5];
int front = -1;
int rear = -1;
void enqueue(int n)
{
    if (rear == 4)
    {
        cout << "Queue is full";
        return;
    }

    if (front == -1)
        front = 0;

    rear++;
    queue[rear] = n;
}

void deq()
{
    if (front == -1 || front > rear)
    {
        cout << "Queue is empty";
        return;
    }

    front++;
}

void show()
{
    if (front == -1 || front > rear)
    {
        cout << "Queue is empty";
        return;
    }

    for (int i = front; i <= rear; i++)
    {
        cout << queue[i] << " ";
    }
}

int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);
    enqueue(40);
    enqueue(50);
```

```
show();

deq();
cout << "\n===== \n";
show();
return 0;
}
```

Output:

```
10 20 30 40 50
=====
20 30 40 50

...Program finished with exit code 0
```

b. Simulate a simple queuing system (e.g., customer service queue).

Code:

```
#include <iostream>
#include <string>
using namespace std;

string queue[5];
int front = -1;
int rear = -1;

void enqueue(string name)
{
    if (rear == 5 - 1)
    {
        cout << "Queue is full!";
        return;
    }

    if (front == -1)
        front = 0;

    rear++;
    queue[rear] = name;
}

void dequeue()
{
    if (front == -1 || front > rear)
    {
        cout << "Queue is empty!";
        return;
    }

    cout << "Serving customer:" << queue[front] << endl;
    front++;
}

void show()
{
    if (front == -1 || front > rear)
    {
        cout << "Queue is empty!";
        return;
    }

    for (int i = front; i <= rear; i++)
    {
        cout << queue[i] << " ";
    }
}

int main()
{
    enqueue("om");
    enqueue("Amit");
    enqueue("mahesh");
```

```
enqueue("chinmay");
enqueue("Rohan");

cout << "Customers in queue:\n";
show();

dequeue();

cout << "\n===== \n";

cout << "Customers after serving:\n";
show();

return 0;
}
```

Output:

```
Customers in queue:
om Amit mahesh chinmay Rohan Serving customer:om

=====
Customers after serving:
Amit mahesh chinmay Rohan

...Program finished with exit code 0
```

Practical no:05

5.Binary Search Tree: Write a program to:

a. Create a binary search tree.

Code:

```
#include<iostream>
#include<stdlib.h>
using namespace std;

struct node
{
    int data;
    struct node*left;
    struct node*right;
};

struct node*new_node(int val)
{
    struct node*nn=(struct node*)malloc(sizeof(struct node));
    nn->data=val;
    nn->left=NULL;
    nn->right=NULL;
    return nn;
}

void inorder(struct node*r)
{
    if(r==NULL)
    {
        return;
    }
    inorder(r->left);
    cout<<r->data<<" ";
    inorder(r->right);
}

void preorder(struct node*r)
{
    if(r==NULL)
    {
        return;
    }
    cout<<r->data<<" ";
    preorder(r->left);
    preorder(r->right);
}

void postorder(struct node*r)
{
    if(r==NULL)
    {
        return;
    }
    postorder(r->left);
    postorder(r->right);
    cout<<r->data<<" ";
}
```

```
int main()
{
    struct node*root=new _node(10);
    root->left=new _node(7);
    root->right=new _node(9);
    cout<<"Inorder:";
    inorder(root);
    cout<<"\nPreorder:";
    preorder(root);
    cout<<"\nPostorder:";
    postorder(root);
    return 0;
}
```

Output:

```
Inorder:7 10 9
Preorder:10 7 9
Postorder:7 9 10
```

```
...Program finished with exit code 0
```

Practical No.06

6. Tree Traversal

Write a program to:

a. Implement pre-order, in-order, Post-order traversal of a binary tree.

Code:

```
#include<iostream>
#include<stdlib.h>
using namespace std;

struct node
{
    int data;
    struct node*left;
    struct node*right;
};

struct node*new_node(int val)
{
    struct node*nn=(struct node*)malloc(sizeof(struct node));
    nn->data=val;
    nn->left=NULL;
    nn->right=NULL;
    return nn;
}

void inorder(struct node*r)
{
    if(r==NULL)
    {
        return;
    }

    inorder(r->left);
    cout<<r->data<<" ";
    inorder(r->right);
}

void preorder(struct node*r)
{
    if(r==NULL)
    {
        return;
    }

    cout<<r->data<<" ";
    preorder(r->left);
    preorder(r->right);
}

void postorder(struct node*r)
{
    if(r==NULL)
    {
        return;
    }
```

```
}

postorder(r->left);
postorder(r->right);
cout<<r->data<<" ";
}

int main()
{
    struct node*root=new _node(10);
    root->left=new _node(7);
    root->right=new _node(9);

    cout<<"Inorder: ";
    inorder(root);

    cout<<"\nPreorder: ";
    preorder(root);

    cout<<"\nPostorder: ";
    postorder(root);

    return 0;
}
```

Output:

```
Inorder: 8 10 9
Preorder: 10 8 9
Postorder: 8 9 10
```

...Program finished with exit code 0

Practical No.07

7.Hash Table:

- Implement a hash table with separate chaining for collision handling.
- Store and retrieve data from the hash table.

```
#include<iostream>
#include<stdlib.h>
using namespace std;
struct node
{
int data;
struct node*next;
};
int main()
{
int values[]={23,75,51,90,91},key,index,i,size=5,vsize;
struct node*hash_table[size]={NULL};
vsize=sizeof(values)/sizeof(values[0]);
for(i=0;i<vsize;i++)
{
key=values[i];
index=key%size;
struct node*nn=(struct node*)malloc(sizeof(struct node));
nn->data=key;
nn->next=hash_table[index];
hash_table[index]=nn;
}
for(i=0;i<size;i++)
{
cout<<i<<":";
struct node*tr=hash_table[i];
while(tr!=NULL)
{
cout<<tr->data<<"-";
tr=tr->next;
}
cout<<"NULL"<<endl;
}
int sk=23,f=0;
for(i=0;i<size;i++)
{
index=sk%size;
struct node*t=hash_table[index];
while(t!=NULL)
{
if(t->data==sk)
{
f=1;
break;
}
t=t->next;
}
}
if(f==1)
{
```

```
cout<<"\nElement found";
}
else
{
cout<<"\nElement not found";
}
return 0;
}
```

Output:

0:90->75->NULL

1:91->51->NULL

2:NULL

3:23->NULL

4:NULL

Element found

...Program finished with exit code 0

Practical No.09

9.Searching Algorithms:

Write programs to implement and compare:

a. Linear search

b. Binary search (on a sorted array)

Code:

```
#include<iostream>
```

```
using namespace std;
```

```
int a[]={12,56,88,90,340},i,mid,l,r,siz,sk,f=0;
```

```
void b_s(int a[],int siz,int sk)
```

```
{
```

```
l=0;
```

```
r=siz-1;
```

```
while(l<=r)
```

```
{
```

```
mid=(l+r)/2;
```

```
if(sk==a[mid])
```

```
{
```

```
f=1;
```

```
break;
```

```
}
```

```
else if(sk<a[mid])
```

```
{
```

```
r=mid-1;
```

```
}
```

```
else
```

```
{
```

```
l=mid+1;
```

```
}
```

```
}
```

```
if(f==1)
```

```
{
```

```
cout<<"Element found";
```

```
}
```

```
else
```

```
{
```

```
cout<<"Element not found";
```

```
}
```

```
}
```

```
int main()
```

```
{
```

```
siz=sizeof(a)/sizeof(a[1]);
```

```
cout<<"Enter number: ";
```

```
cin>>sk;
```

```
b_s(a,siz,sk);
```

```
return 0;
```

```
}
```

Output:

Enter number: 18
Element not found

...Program finished with exit code 0

Enter number: 340
Element found

...Program finished with exit code 0